EXPERIMENT 1

```
class User:
    def __init__(self, user_id, name):
        self.user_id = user_id
        self.name = name

    def display_details(self):
        print(f"User ID: {self.user_id}, Name: {self.name}\n")

# Customer class
class Customer(User):
    def __init__(self, user_id, name):
        super().__init__(user_id, name)
        self.orders = []

    def place_order(self, restaurant, items_selected):
        for choice in items_selected:
            if 1 <= choice <= len(restaurant.menu):
                item, price = list(restaurant.menu.items())[choice - 1]
                order = Order(self, restaurant, item, price)
                self.orders.append(order)
                print(f"{self.name} placed an order for {item} from {restaurant.name}\n")
            else:
                print(f"Invalid choice {choice}, skipping...\n")

# Premium Customer
class PremiumCustomer(Customer):
    def calculate_discount(self, amount):
```

```
        discount = amount * 0.2
        return amount - discount

# Delivery Partner
class DeliveryPartner:
    def __init__(self, partner_id, name):
        self.partner_id = partner_id
        self.name = name

    def assign_delivery(self, order):
        print(f"Delivery Partner {self.name} assigned to deliver {order.item}\n")

# Restaurant class
class Restaurant:
    def __init__(self, rest_id, name, type, menu):
        self.rest_id = rest_id
        self.name = name
        self.type = type
        self.menu = menu

    def display_menu(self):
        print(f"\nRestaurant: {self.name} ({self.type})")
        print("Menu:")
        for idx, (item, price) in enumerate(self.menu.items(), start=1):
            print(f"{idx}. {item} - ₹{price}")
        print("\n")

# Order class
```

```
class Order:
    def __init__(self, customer, restaurant, item, price):
        self.customer = customer
        self.restaurant = restaurant
        self.item = item
        self.price = price

    def generate_bill(self):
        if isinstance(self.customer, PremiumCustomer):
            final_amount = self.customer.calculate_discount(self.price)
            print(
                f"Bill for Premium Customer {self.customer.name}: "
                f"₹{final_amount}\n"
            )
            return final_amount
        else:
            print(
                f"Bill for Customer {self.customer.name}: "
                f"₹{self.price}\n"
            )
            return self.price

# Restaurants
r1 = Restaurant("R101", "Green Leaf", "Veg", {
    "Paneer Butter Masala": 180,
    "Veg Biryani": 150,
    "Dal Tadka": 120
})
```

```
r2 = Restaurant("R102", "Meat Lovers", "Non-Veg", {
    "Chicken Biryani": 250,
    "Mutton Curry": 300,
    "Fish Fry": 220
})

r3 = Restaurant("R103", "Mixed Flavors", "Veg & Non-Veg", {
    "Veg Burger": 100,
    "Chicken Burger": 150,
    "French Fries": 80
})

restaurants = [r1, r2, r3]

# Display restaurants
print("\nAvailable Restaurants:")

for idx, rest in enumerate(restaurants, start=1):
    print(f"{idx}. {rest.name} ({rest.type})")

print("\n")

cust_type = input("Enter customer type (normal/premium): ").lower()
cust_name = input("Enter customer name: ")
cust_id = input("Enter customer ID: ")

print("\n")
```

```
if cust_type == "premium":
    customer = PremiumCustomer(cust_id, cust_name)
else:
    customer = Customer(cust_id, cust_name)

# Select restaurant
rest_choice = int(input("Enter restaurant number to order from: "))

print("\n")

if 1 <= rest_choice <= len(restaurants):

    selected_rest = restaurants[rest_choice - 1]

    selected_rest.display_menu()

    # Multiple item selection
    items = input("Enter item numbers separated by commas: ")

    items_selected = [int(x.strip()) for x in items.split(",")]

    print("\n")

    customer.place_order(selected_rest, items_selected)

    # Assign delivery partner
    partner = DeliveryPartner("DP101", "Ramesh")
```

```
    total_bill = 0

    print("\n--- CART SUMMARY ---")

    for order in customer.orders:

        print(
            f"{order.item} from "
            f"{order.restaurant.name} - ₹{order.price}"
        )

        partner.assign_delivery(order)

        total_bill += order.generate_bill()

    print(f"Final Total Bill: ₹{total_bill}\n")

else:
    print("Invalid restaurant choice!\n")
```

OUTPUT:

```
Available Restaurants:
1. Green Leaf (Veg)
2. Meat Lovers (Non-Veg)
3. Mixed Flavors (Veg & Non-Veg)

Enter customer type (normal/premium): NORMAL
Enter customer name: NIKHIL
```

Enter customer ID: 1234

Enter restaurant number to order from: 2

Restaurant: Meat Lovers (Non-Veg)

Menu:

1. Chicken Biryani - ₹250

2. Mutton Curry - ₹300

3. Fish Fry - ₹220

Enter item numbers separated by commas: 1,2

NIKHIL placed an order for Chicken Biryani from Meat Lovers

NIKHIL placed an order for Mutton Curry from Meat Lovers

--- CART SUMMARY ---

Chicken Biryani from Meat Lovers - ₹250

Delivery Partner Ramesh assigned to deliver Chicken Biryani

Bill for Customer NIKHIL: ₹250

Mutton Curry from Meat Lovers - ₹300

EXPERIMENT 2

```
class BankAccount:
    def __init__(self,account_no,name,balance=0):
        self.__account_no = account_no
        self.__balance = balance
        self.__name = name
        self.__pin=None

    def get_balance(self):  #public getter method
        return self.__balance

    def set_pin(self,pin):   #public setter method
        if len(str(pin))==4:
            self.__pin=pin
            return True
        return False

    def __add__(self, other):
        total= self.__balance + other.get_balance()
        return f"Total Combined: Rs.{total}"

    def __gt__(self, other):
        return self.__balance > other.get_balance()

class SavingsAccount(BankAccount):
    def calculate_interest(self):
        return self.get_balance()*0.04

class CurrentAccount(BankAccount):
    def calculate_interest(self):
        return self.get_balance()*0.01
```

```
class FixedDepositAccount(BankAccount):
    def calculate_interest(self):
        return self.get_balance()*0.07

savings=SavingsAccount(12345,"John Doe",10000)
current=CurrentAccount(67890,"Jane Smith",2000)
fd=FixedDepositAccount(54321,"Alice Johnson",50000)
accounts=[savings,current,fd]
for account in accounts:
    print(account.calculate_interest())

acc1=SavingsAccount(11111,"Bob Brown",15000)
acc2=CurrentAccount(22222,"Charlie Davis",5000)
print(acc1 + acc2)
print(acc1 > acc2)
print(acc1 < acc2)
```

OUTPUT:

400.0

20.0

3500.0000000000005

Total Combined: Rs.20000

True

False

EXPERIMENT 3

```
class Vehicle:
    def __init__(self, vehicle_id, model, category, rental_rate):
        self.vehicle_id = vehicle_id
        self.model = model
        self.category = category
        self.rental_rate = rental_rate

    def __str__(self):
        return f"{self.category}: {self.model} (ID: {self.vehicle_id})"

    def __repr__(self):
        return f"Vehicle('{self.vehicle_id}', '{self.model}', '{self.category}', {self.rental_rate})"

class Car(Vehicle):
    def __init__(self, vehicle_id, model, rental_rate):
        super().__init__(vehicle_id, model, "Car", rental_rate)

class Bike(Vehicle):
    def __init__(self, vehicle_id, model, rental_rate):
        super().__init__(vehicle_id, model, "Bike", rental_rate)

class ElectricScooter(Vehicle):
    def __init__(self, vehicle_id, model, rental_rate):
        super().__init__(vehicle_id, model, "Electric Scooter", rental_rate)

class SUV(Vehicle):
    def __init__(self, vehicle_id, model, rental_rate):
```

```
        super().__init__(vehicle_id, model, "SUV", rental_rate)

class Rental:
    def __init__(self, customer_name, vehicle, days, extra_km=0):
        self.customer_name = customer_name
        self.vehicle = vehicle
        self.days = days
        self.extra_km = extra_km
        self.security_deposit = 500

    def calculate_rent(self):
        # Weekend rate is 20% higher
        base_rent = self.vehicle.rental_rate * self.days

        # Additional kilometer charge
        extra_charge = self.extra_km * 5

        total = base_rent + extra_charge + self.security_deposit
        return total

    def __len__(self):
        return self.days

    def __str__(self):
        return f"Rental for {self.customer_name}: {self.vehicle}"

    def __repr__(self):
        return f"Rental('{self.customer_name}', {repr(self.vehicle)}, {self.days})"
```

```
    def __del__(self):
        print("Rental object deleted")

# Creating different vehicle objects
car = Car("C101", "Toyota Camry", 1500)
bike = Bike("B101", "Royal Enfield", 800)
scooter = ElectricScooter("E101", "Ather 450X", 600)
suv = SUV("S101", "Toyota Fortuner", 2500)

# Creating rental
rental = Rental("Nikhil", car, 3, 20)

# Display details
print("----- VEHICLE DETAILS -----")
print(car)
print(bike)
print(scooter)
print(suv)

print("\n----- RENTAL DETAILS -----")
print(rental)

print("\nCustomer:", rental.customer_name)
print("Vehicle:", rental.vehicle.model)
print("Category:", rental.vehicle.category)
print("Rental Days:", len(rental))
print("Extra Kilometers:", rental.extra_km)
```

```
print("Security Deposit:", rental.security_deposit)

print("\n----- RENT CALCULATION -----")
print("Base Rent:", rental.vehicle.rental_rate * rental.days)
print("Extra KM Charge:", rental.extra_km * 5)
print("Security Deposit:", rental.security_deposit)
print("Total Rent:", rental.calculate_rent())

print("\n----- REPR OUTPUT -----")
print(repr(car))
print(repr(rental))

# Demonstrating object deletion
del rental
```

OUTPUT:

```
----- VEHICLE DETAILS -----
Car: Toyota Camry (ID: C101)
Bike: Royal Enfield (ID: B101)
Electric Scooter: Ather 450X (ID: E101)
SUV: Toyota Fortuner (ID: S101)

----- RENTAL DETAILS -----
Rental for Nikhil: Car: Toyota Camry (ID: C101)

Customer: Nikhil
Vehicle: Toyota Camry
```

Category: Car

Rental Days: 3

Extra Kilometers: 20

Security Deposit: 500

----- RENT CALCULATION -----

Base Rent: 4500

Extra KM Charge: 100

Security Deposit: 500

Total Rent: 5100

----- REPR OUTPUT -----

Vehicle('C101', 'Toyota Camry', 'Car', 1500)

Rental('Nikhil', Vehicle('C101', 'Toyota Camry', 'Car', 1500), 3)

Rental object deleted